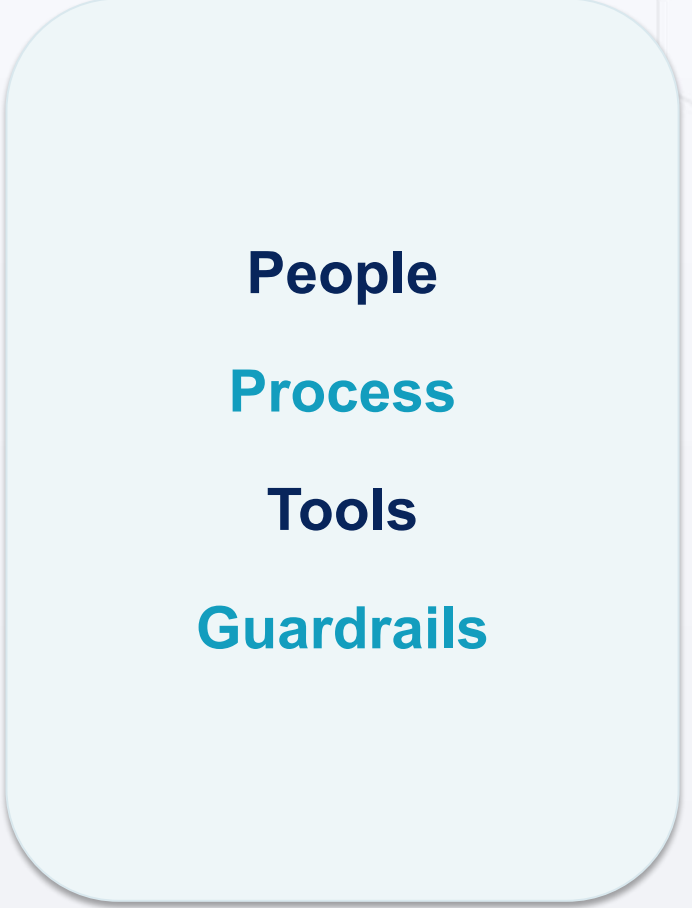




Generative AI for Software Teams

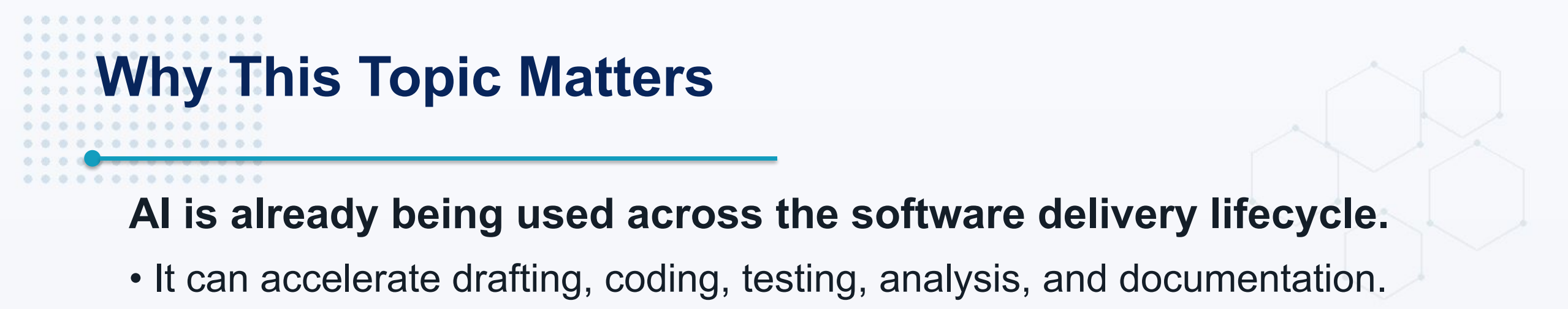
Prompt engineering, tools, coding agents, privacy, risks, and practical guardrails

Amjad Hossain
28th June, 2026



People
Process
Tools
Guardrails

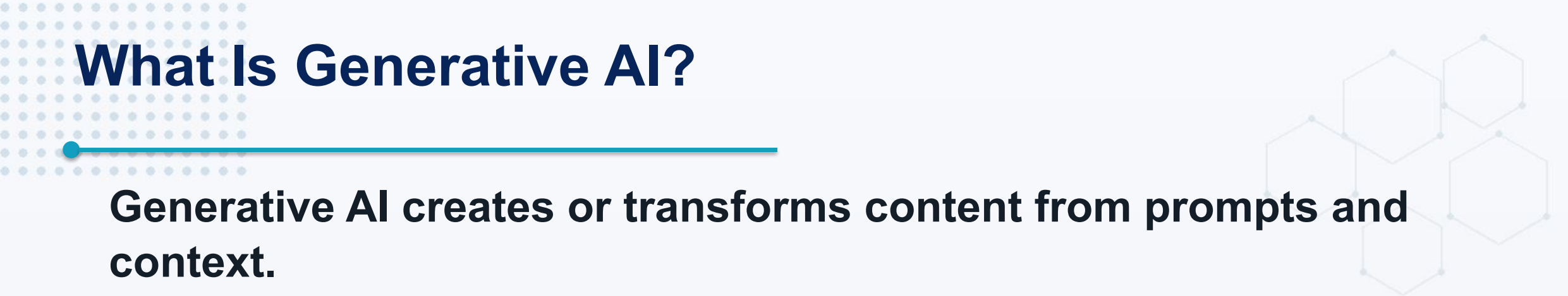
Why This Topic Matters



AI is already being used across the software delivery lifecycle.

- It can accelerate drafting, coding, testing, analysis, and documentation.
 - But speed without guardrails can create security, quality, and privacy problems.
 - The real skill is not only using AI. It is using AI safely, deliberately, and productively.
- 

What Is Generative AI?



Generative AI creates or transforms content from prompts and context.

- It can generate text, code, summaries, test cases, images, explanations, and structured output.
 - Most modern tools are powered by large language models and related multimodal models.
 - For software teams, Generative AI is best seen as an accelerator, not as an authority.
- 

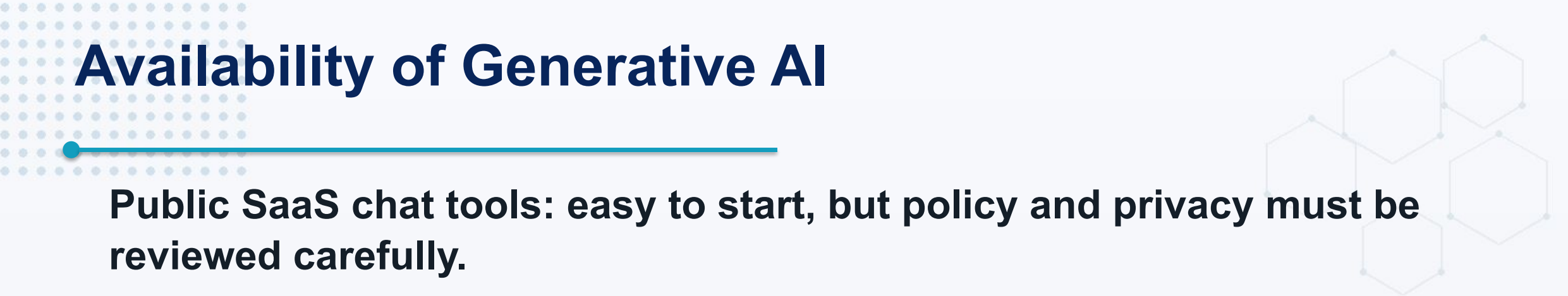
Main Purpose of Generative AI



Brainstorming and ideation

- Drafting and rewriting text
 - Summarization and extraction
 - Coding, refactoring, and test generation
 - Reasoning support and troubleshooting
 - Multimodal work with text, image, and sometimes audio
- 

Availability of Generative AI



Public SaaS chat tools: easy to start, but policy and privacy must be reviewed carefully.

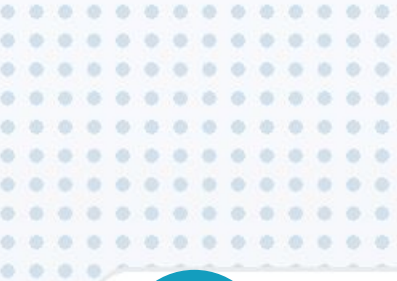
- IDE assistants: integrated help for code generation, explanation, and refactoring.
 - APIs: use AI inside your own applications and automation.
 - Enterprise managed offerings: stronger admin control, auditability, and privacy options.
 - Private or self-hosted/open-weight options: more control, but more operational responsibility.
- 

What AI Is Good At — And Not Good At



Good at speed, first drafts, pattern matching, summarization, and repetitive work.

- Good at helping humans explore options, compare approaches, and unblock work.
 - Weak at guaranteeing truth, completeness, current policy interpretation, and hidden business context.
 - Therefore: trust but verify, and never remove human accountability.
- 



1

Prompt Engineering

Better prompts do not magically make AI correct, but they usually make AI much more useful.

**Practical Training
for Software Teams**



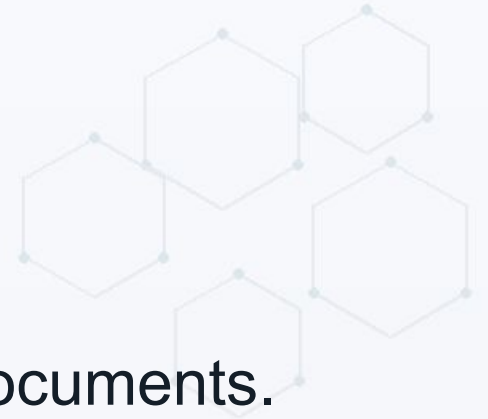
Prompt Engineering Basics



State the goal clearly.

- Provide the right context.
 - Define the expected output format.
 - Give constraints, boundaries, and acceptance criteria.
 - Iterate when the first answer is not good enough.
- 

A Practical Prompt Structure



Task: what exactly should the AI do?

- Context: project, domain, architecture, audience, or relevant documents.
 - Constraints: language, framework, security rules, style, and scope limits.
 - Output: bullet list, table, JSON, code, or step-by-step explanation.
 - Quality bar: ask it to explain assumptions, trade-offs, risks, and missing information.
- 

Prompt Engineering Best Practices



Be specific; vague prompts create vague answers.

- Give examples when the output shape matters.
 - Break complex tasks into smaller steps.
 - Ask for reasoning aids like assumptions, trade-offs, and test ideas.
 - Tell the model when to say “I do not know” or when more data is needed.
 - Use iteration: draft → critique → improve → verify.
- 

Common Prompt Anti-Patterns



“Build everything” with no scope or constraints.

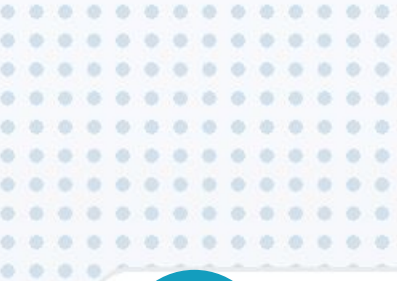
- Giving confidential data to a tool without checking policy.
 - Asking for code without asking for tests, edge cases, and security concerns.
 - Treating a single answer as final truth.
 - Using AI for legal, compliance, or architecture decisions without human review.
- 

Reference Learning Resources



ChatGPT Prompt Engineering for Developers — DeepLearning.AI

- Taught by Isa Fulford and Andrew Ng.
 - Focus: prompt engineering best practices, iterative prompting, summarizing, inferring, transforming, expanding, and chatbot building.
 - AI Prompting for Everyone — DeepLearning.AI
 - Taught by Andrew Ng.
 - Focus: finding information, web search, deep research, brainstorming, writing, multimedia, and working with code.
- 



2

Tools and Data Privacy

In practice, data privacy usually depends more on plan type and configuration than on brand name alone.



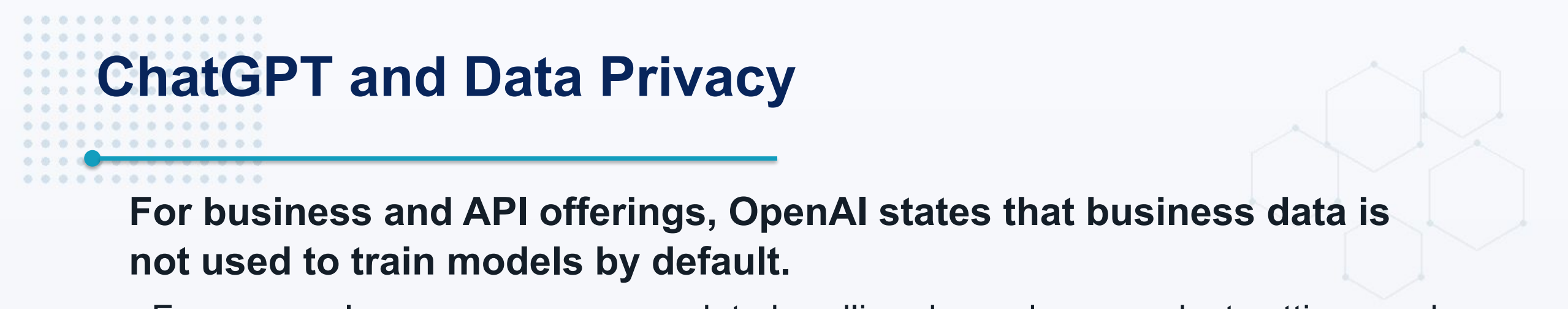
**Practical Training
for Software Teams**



AI Tool Landscape

Tool	Primary Strength	Typical Use in Teams
ChatGPT	General-purpose assistant	Writing, analysis, brainstorming, code help, research, image tasks
GitHub Copilot	IDE-native coding assistance	Inline suggestions, chat, code explanation, refactoring, tests
Claude	Strong writing and reasoning support	Analysis, long-form synthesis, drafting, coding support
Codex	Coding agent for software work	Task execution, code changes, debugging, test-driven flows
Claude Code	Terminal/agentic coding workflow	Repo-aware coding help, implementation, review, iteration

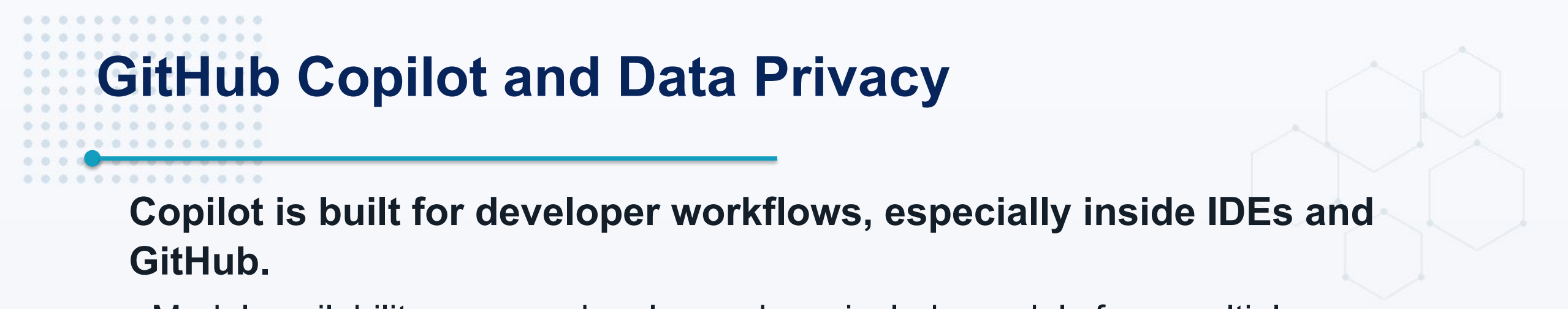
ChatGPT and Data Privacy



For business and API offerings, OpenAI states that business data is not used to train models by default.

- For personal or consumer usage, data handling depends on product settings and policy.
 - Good use: ideation, summarization, documentation, code explanation, and controlled coding tasks.
 - Bad use: pasting secrets, production data, regulated documents, or internal credentials without approval.
- 

GitHub Copilot and Data Privacy



Copilot is built for developer workflows, especially inside IDEs and GitHub.

- Model availability can vary by plan and can include models from multiple providers.
 - Business and Enterprise plans have stronger policy commitments than personal use.
 - For software teams, Copilot should still be treated as a governed tool, not an unrestricted code source.
- 

Claude and Data Privacy



Claude is widely used for reasoning, writing, and coding assistance.

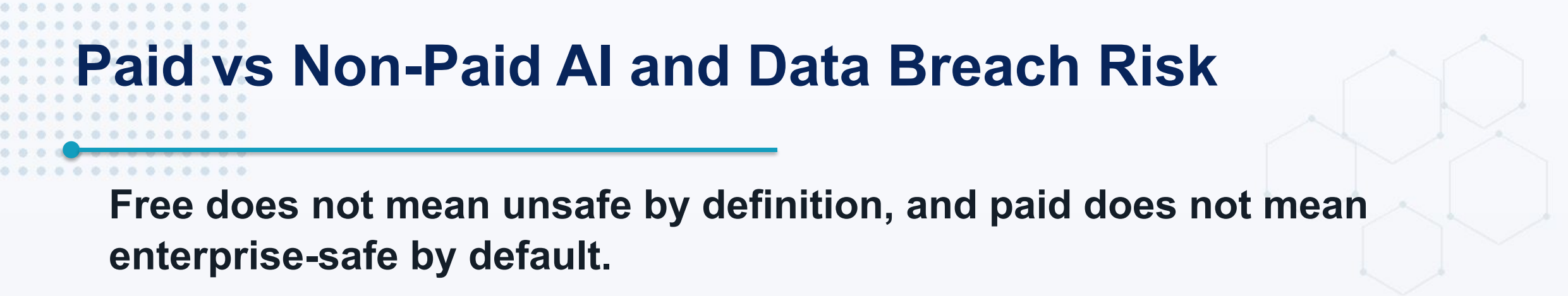
- Commercial and enterprise offerings emphasize governance and admin controls.
 - Consumer and personal use may have different privacy behavior than enterprise use.
 - Always verify what the current plan says about training, retention, and admin control.
- 

Practical Privacy Comparison

Policies change over time. Always confirm the current official terms before rollout.

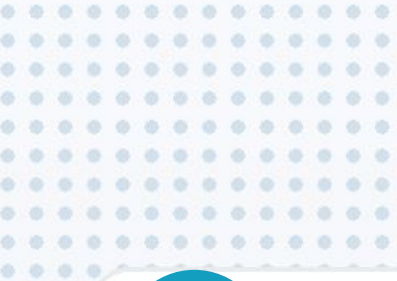
Plan / Context	Typical Privacy Posture	Recommendation
Free or consumer tools	Convenient, but higher governance risk	Do not paste sensitive code, secrets, or regulated data
Paid personal tools	Often better limits or features, but not equal to enterprise governance	Treat as semi-trusted unless policy explicitly approves use
Business / Enterprise plans	Usually stronger admin control, auditability, and training protections	Preferred for company rollout after legal/security review
API with approved architecture	More controllable inside your own workflows	Best for governed internal automation and product features

Paid vs Non-Paid AI and Data Breach Risk



Free does not mean unsafe by definition, and paid does not mean enterprise-safe by default.

- The real question is: what does the specific plan allow, retain, or use for model improvement?
 - Paid personal plans may still be unsuitable for confidential company data.
 - Enterprise or approved API usage is usually the safer path for software organizations.
 - Never assume privacy. Verify policy, settings, retention, and admin controls.
- 



3

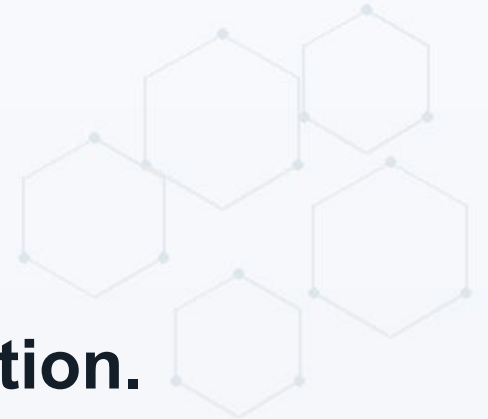
AI Agents for Daily Development

An AI agent is more than a chat response. It can plan, modify files, run tools, and iterate toward a result.

**Practical Training
for Software Teams**



What Is an AI Agent?



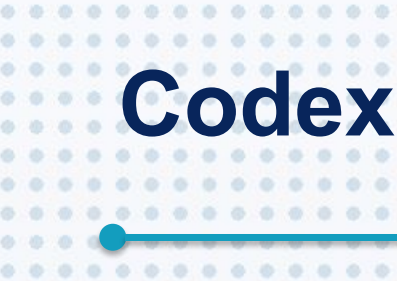
An AI agent can take a goal, use tools, inspect code or documents, and perform multiple steps toward completion.

- Examples of tools it may use: file editing, terminal commands, search, tests, linting, or environment inspection.
 - This makes agents powerful for software work — and also more risky if not constrained.
- 

Coding Agents at a Glance

Agent	Strength	Caution
Codex	Strong coding workflow and agentic execution	Still needs review, tests, and security gates
GitHub Copilot	Excellent IDE integration and daily coding support	Can over-suggest or drift without clear task framing
Claude Code	Strong terminal/repo-based reasoning and implementation	May produce plausible but incomplete changes without domain context

Codex for Daily Development



Useful for implementing scoped tasks, exploring a codebase, generating tests, and fixing issues.

- Model choice matters: stronger reasoning models are useful for complex tasks; lighter models are useful for quick iterations.
 - Best use: give clear scope, constraints, and success criteria; then verify changes through tests and code review.
- 

GitHub Copilot for Daily Development



Best for inline suggestions, code explanation, small refactors, and working inside the developer flow.

- Useful for generating unit tests, documentation comments, and transformation code.
 - Works best when the repository structure, naming, and conventions are already clean.
- 

Claude Code for Daily Development



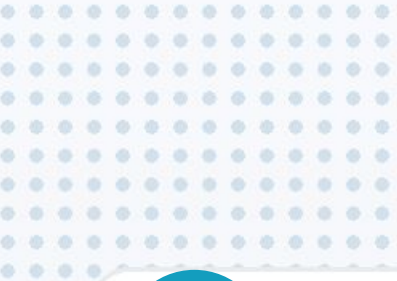
Useful when you want stronger analysis around architecture, trade-offs, code reasoning, and multi-step changes.

- Helpful for exploring unfamiliar repositories, drafting implementation plans, and reviewing risky changes.
 - Like every agent, it must still be constrained by policy, context, and review.
- 

Choosing Model Types for Daily Work

Model names change frequently. Choose by capability, speed, cost, and risk — not by hype.

Task Type	Preferred Model Characteristic	Example Guidance
Autocomplete / quick edits	Fast, low-latency model	Use cheaper/faster models for short repetitive tasks
Normal coding and tests	Balanced coding model	Good default for most daily work
Architecture / deep debugging	Stronger reasoning model	Use frontier or high-effort models for difficult tasks
Bulk transformations	Cost-efficient model with strong formatting discipline	Great for repetitive migrations or doc updates
Risky production changes	Best reasoning model plus human review	Do not optimize only for speed when the blast radius is high



4

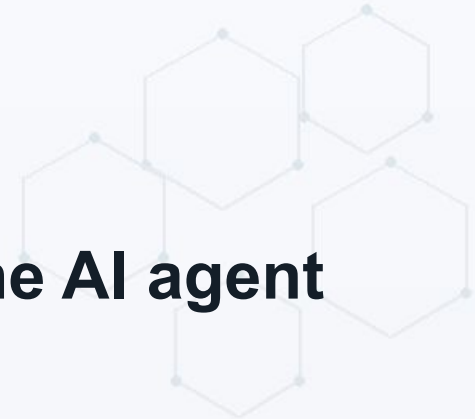
SKILL.md for AI Agents

A good AI agent becomes much more reliable when the project tells it how the team works.

**Practical Training
for Software Teams**



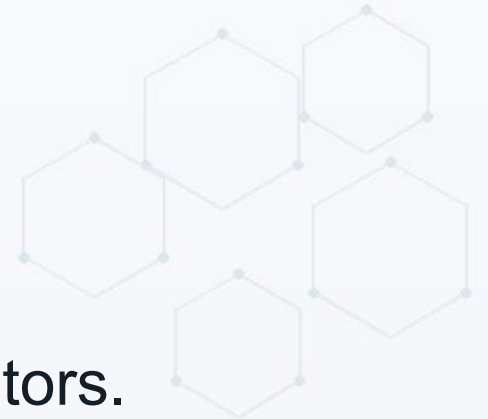
What Is SKILL.md?



SKILL.md is a project-level instruction file that teaches the AI agent how to work inside a specific repository or team context.

- It can describe architecture, naming rules, coding style, test expectations, security boundaries, and review criteria.
 - Its purpose is not magic. Its purpose is consistency, context, and reduced ambiguity.
- 

Why SKILL.md Helps



Reduces repeated prompting.

- Improves consistency across multiple AI sessions and contributors.
 - Helps the agent understand domain rules and non-functional requirements.
 - Encourages safer output because constraints are explicit.
 - Makes review easier because expectations are documented.
- 

What to Put Inside SKILL.md



Project purpose and domain context

- Architecture boundaries and important folders
 - Coding standards and naming conventions
 - Testing rules and minimum coverage expectations
 - Security rules, secret handling, and forbidden actions
 - Definition of done and review checklist
- 

SKILL.md Example for a New Endpoint (Part 1)

SKILL.md

```
# SKILL.md

## Service
Orders Microservice

## Tech Stack
- Spring boot for Backend
- Angular for Frontend
- Oracle as DB

## Task
Expose a new endpoint: POST /api/v1/orders/{orderId}/cancel

## Requirements
- Use existing controller/versioning pattern
- Validate route and request body
- Require authenticated internal user with Orders.Cancel permission
- Publish domain event OrderCancelled
- Return ProblemDetails for errors
- Add unit tests and integration tests
```

**Use this as context
before asking Copilot
or another coding
agent to implement
the work.**

- The file gives the agent business, architectural, and quality boundaries.

SKILL.md Example for a New Endpoint (Part 2)

SKILL.md

Constraints

- Do not bypass MediatR/CQRS flow
- Do not access DbContext from controller
- Do not log PII or secrets
- Follow OpenAPI and existing response style

Done Criteria

- Endpoint added
- Handler, validator, tests complete
- Swagger updated
- Security and authorization checked
- Build and tests pass

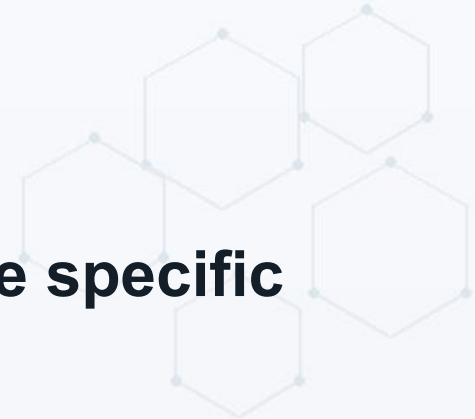
Copilot Prompt

Read SKILL.md and implement the endpoint accordingly.
Propose the plan first, then generate code and tests.

A concise “Copilot Prompt” can tell the agent to follow the file and propose a plan first.

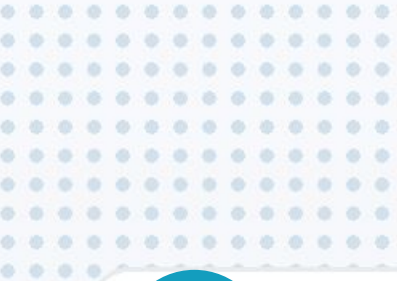
- This reduces random code generation and increases consistency.

Using Copilot with SKILL.md



Step 1: create or update SKILL.md with project rules and the specific task.

- Step 2: ask Copilot to read the repository and follow SKILL.md.
 - Step 3: request an implementation plan before code generation if the task is non-trivial.
 - Step 4: generate code, then immediately request tests, edge cases, and security review points.
 - Step 5: validate everything in CI and human review before merge.
- 



5

AI Across QA, Business, DevOps, and DevSecOps

The value of AI is not limited to code generation. It can support many roles when used with discipline.



**Practical Training
for Software Teams**



AI for QA



Generate test ideas from requirements and user stories.

- Draft unit, integration, API, and UI test cases.
 - Help identify edge cases, invalid inputs, and negative scenarios.
 - Summarize bug reports and propose likely reproduction paths.
 - But do not assume generated tests are sufficient without human test design.
- 

AI for Business and Product Work



Convert rough ideas into structured requirements or user stories.

- Summarize stakeholder notes and meeting discussions.
 - Generate acceptance criteria, FAQ drafts, and release notes.
 - Compare solution options and highlight assumptions or unanswered questions.
- 

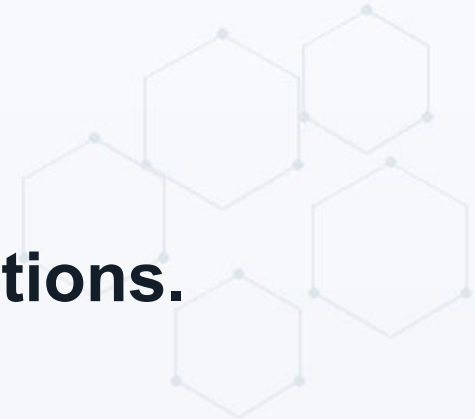
AI for DevOps



Draft CI/CD pipeline templates and infrastructure snippets.

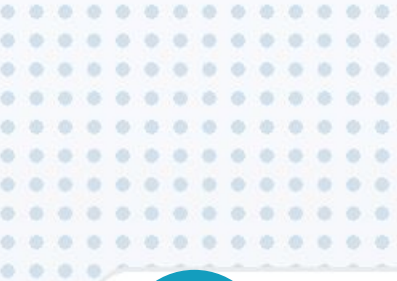
- Explain logs, incidents, deployment failures, and likely causes.
 - Generate runbooks, operational checklists, and post-incident summaries.
 - Help compare deployment strategies, rollback options, and observability gaps.
- 

AI for DevSecOps



Generate secure-by-default checklists and review questions.

- Explain common vulnerability patterns and remediation ideas.
 - Help create threat-model prompts, SAST review checklists, and secrets-handling guidance.
 - But never let AI alone approve security posture or compliance claims.
- 



6

Hallucination, Security, and Unsafe Use

The biggest mistake is not that AI makes errors. The biggest mistake is using AI output without controls.

**Practical Training
for Software Teams**



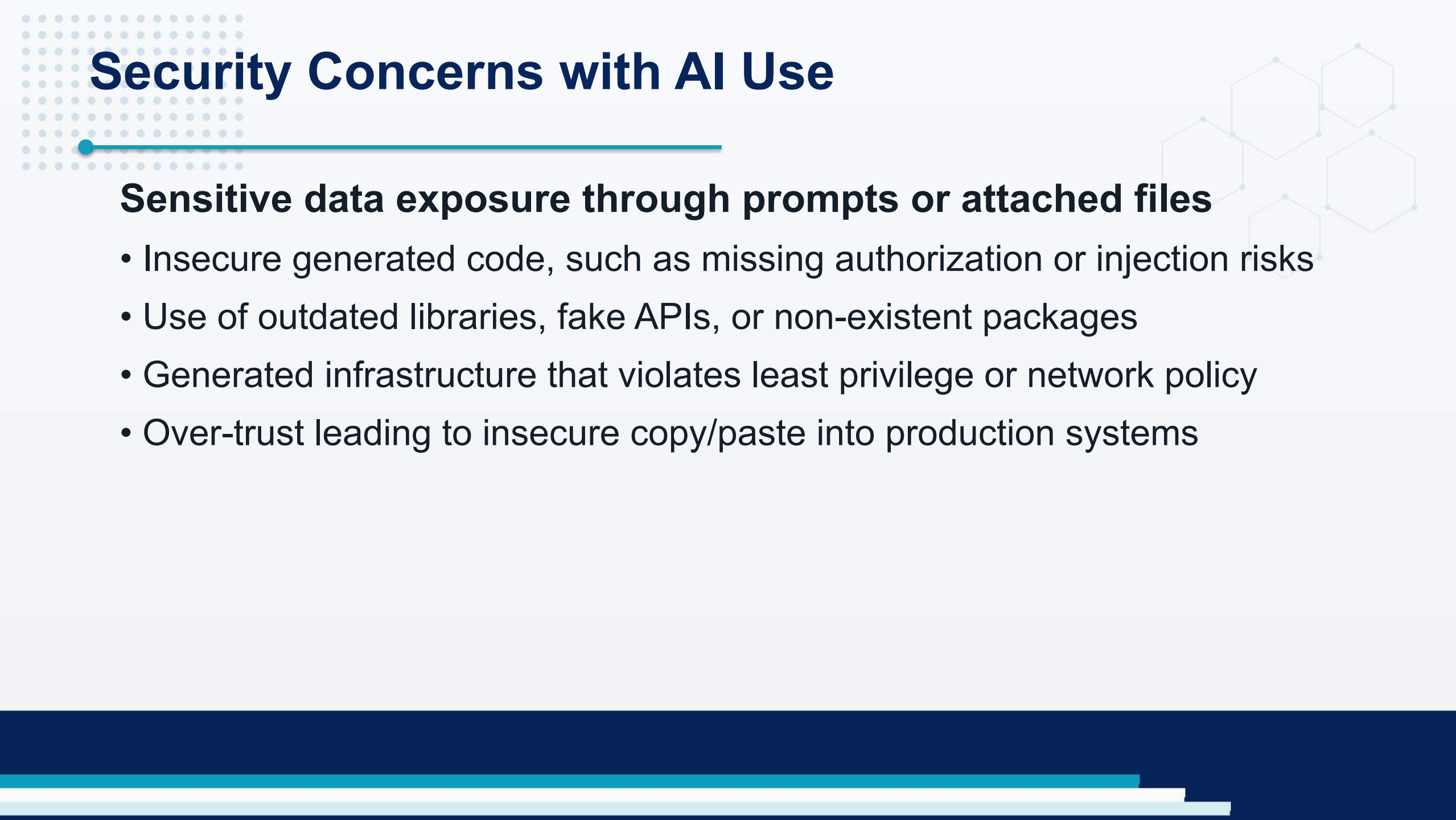
What Is Hallucination?



Hallucination means the AI produces information that sounds correct but is false, incomplete, or invented.

- It can happen in explanations, code, package names, APIs, configuration advice, and security claims.
 - Hallucination is especially dangerous when the output is fluent and confident.
- 

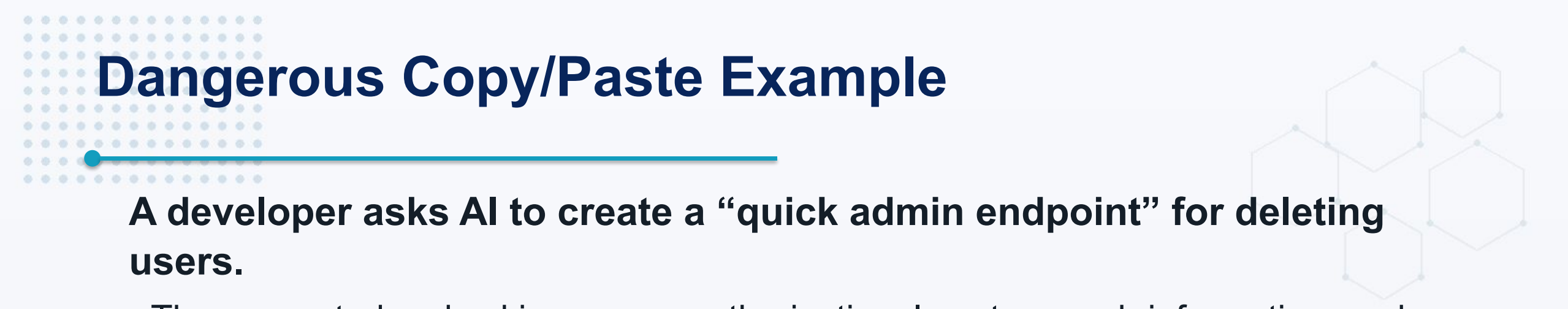
Security Concerns with AI Use



Sensitive data exposure through prompts or attached files

- Insecure generated code, such as missing authorization or injection risks
- Use of outdated libraries, fake APIs, or non-existent packages
- Generated infrastructure that violates least privilege or network policy
- Over-trust leading to insecure copy/paste into production systems

Dangerous Copy/Paste Example



A developer asks AI to create a “quick admin endpoint” for deleting users.

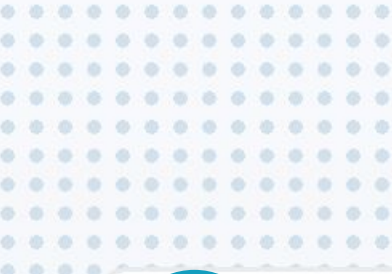
- The generated code skips proper authorization, logs too much information, and performs direct database access.
 - If the developer copies it blindly, the system may now have an IDOR(Insecure **D**irect **O**bject **R**efERENCE), audit gap, and broken architecture rule.
 - The issue is not only the AI output. The issue is skipping review, tests, and security gates.
- 

Do Not Use “Whatever AI Produces”



AI output is a draft, not a production approval.

- Generated code may compile and still be wrong.
 - Generated documentation may sound polished and still mislead teams.
 - Generated YAML or IaC may work and still violate security policy.
 - Human accountability must remain explicit.
- 



7

Best Practices and Guardrails

The winning pattern is simple: useful AI + disciplined humans + automated quality controls.

**Practical Training
for Software Teams**



Best Uses of AI Agents in Software Development



Scaffolding and repetitive code generation

- Test generation and edge-case brainstorming
 - Code explanation and onboarding support
 - Refactoring suggestions and documentation drafting
 - Incident analysis, log summarization, and migration planning
 - Repository-specific work when clear instructions and constraints exist
- 

Security and Quality Gates for AI Output



Human code review

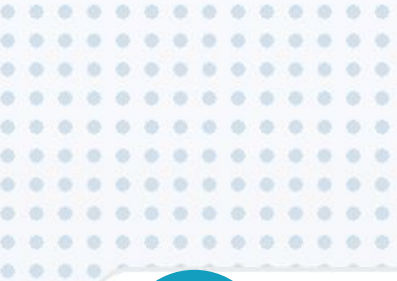
- Unit, integration, and regression tests
 - SAST, SCA, and secrets scanning
 - IaC and container scanning when relevant
 - Architecture and style compliance checks
 - Approval workflow before merge or release
- 

A Recommended Team Operating Model



Approve the tools and plans the team may use.

- Classify what data is allowed, restricted, or forbidden in prompts.
 - Provide standard prompt templates and repository guidance like SKILL.md.
 - Require human review and automated gates for all AI-generated artifacts.
 - Continuously educate the team on privacy, hallucination, and secure usage.
- 



8

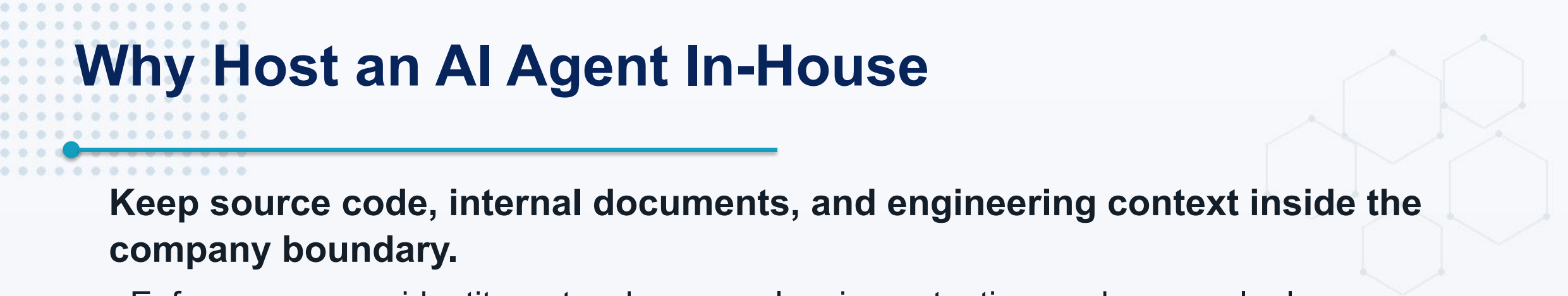
In-House AI Agent Hosting

Self-hosting gives more control, but it also moves infrastructure, security, model lifecycle, and operating cost to the company.

**Private AI Agent
Architecture**



Why Host an AI Agent In-House



Keep source code, internal documents, and engineering context inside the company boundary.

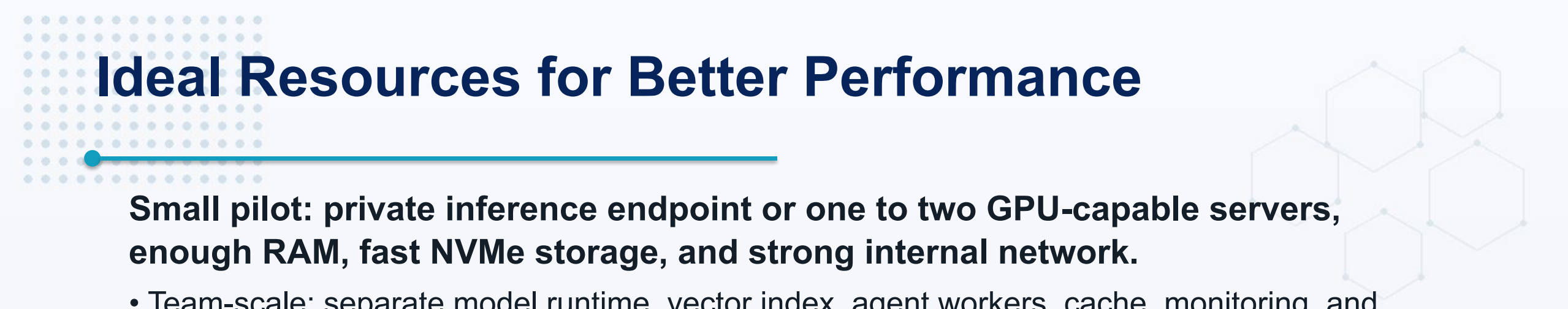
- Enforce company identity, network access, logging, retention, and approval rules.
 - Integrate safely with private Git, CI/CD, artifact repository, ticketing, and documentation systems.
 - Customize behavior through company SKILL.md files, coding standards, and secure prompt templates.
 - Useful when privacy, compliance, latency, or regulatory rules do not allow unrestricted public tools.
- 

In-House AI Agent Reference Architecture



Company Boundary: SSO + MFA, RBAC, network allowlist, DLP, audit logs, rate limits, approvals, and security gates

Ideal Resources for Better Performance



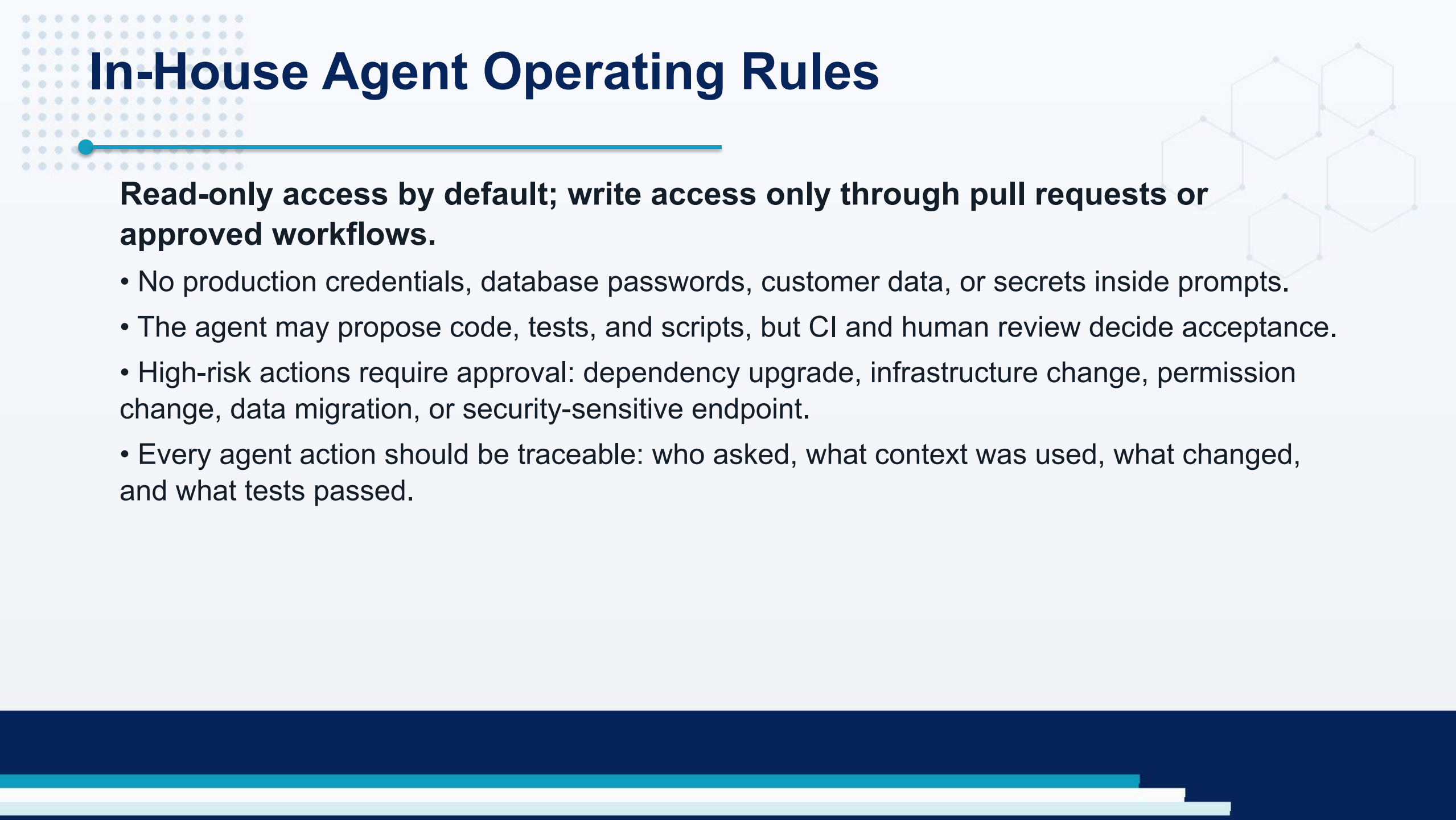
Small pilot: private inference endpoint or one to two GPU-capable servers, enough RAM, fast NVMe storage, and strong internal network.

- Team-scale: separate model runtime, vector index, agent workers, cache, monitoring, and audit storage.
 - Large codebase work: prioritize context indexing, retrieval quality, fast storage, and job queueing, not only model size.
 - High concurrency: plan rate limits, request queues, GPU memory, autoscaling, and fallback model strategy.
 - Do not buy hardware first. Measure workload, model size, concurrency, latency target, security need, and cost.
- 

How to Limit Usage to the Company

Control Area	Recommended Control	Purpose
Identity	SSO, MFA, RBAC, user groups	Only approved employees and teams can access the agent
Network	VPN, private subnet, IP allowlist	Prevent public internet access
Data	DLP, secret scanning, prompt redaction	Block secrets, PII, tokens, and regulated data leakage
Tools	Allowlisted repositories, CI jobs, and commands	Prevent uncontrolled file, terminal, or external tool access
Cost	Quota, rate limit, budget alert	Stop accidental or abusive usage
Audit	Prompt/output logs with retention policy	Support investigation, compliance, and improvement

In-House Agent Operating Rules



Read-only access by default; write access only through pull requests or approved workflows.

- No production credentials, database passwords, customer data, or secrets inside prompts.
- The agent may propose code, tests, and scripts, but CI and human review decide acceptance.
- High-risk actions require approval: dependency upgrade, infrastructure change, permission change, data migration, or security-sensitive endpoint.
- Every agent action should be traceable: who asked, what context was used, what changed, and what tests passed.



8A. In-House Tooling and GPU Sizing

These slides add concrete tool names and practical hardware sizing options for a company-hosted AI agent.



**Tools, Hosting
& Governance**



Installable In-House AI Tool Stack

Tool choice depends on security policy, model choice, user count, and integration needs.

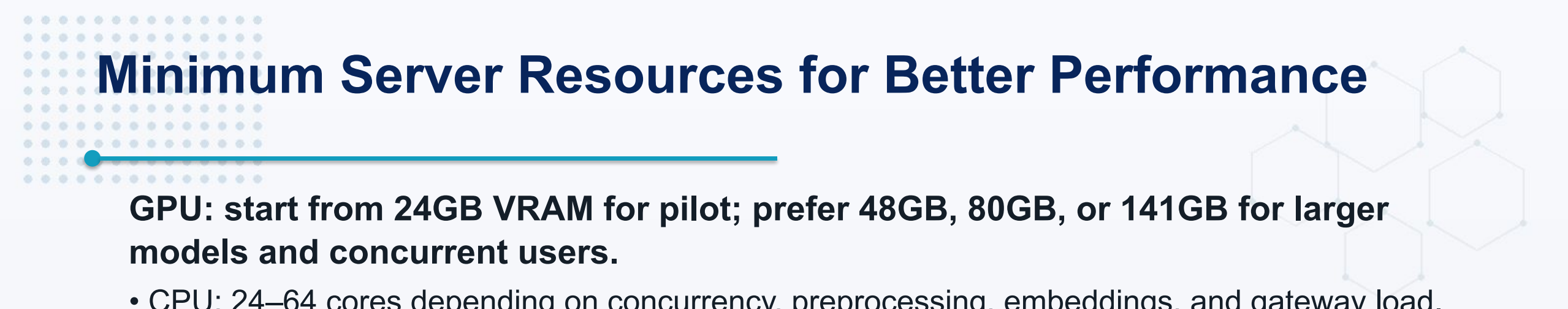
Layer	Sample Tools	Purpose
Model serving	vLLM, Ollama, Hugging Face TGI, llama.cpp server	Host local/open-weight models behind an API
Chat / UI	Open WebUI, LibreChat	Internal browser-based assistant interface
AI gateway	LiteLLM, NGINX/Kong, API gateway	Central routing, auth, logging, quota, model selection
Coding assistant	Tabby, Continue, Aider, OpenHands, Cline/Roo Code	Repo-aware coding help and local/controlled agent workflows
RAG / knowledge	Qdrant, Milvus, pgvector, OpenSearch	Search internal documents, code, ADRs, runbooks, and policies

GPU Size Starting Points

Approximate inference sizing; quantization, context length, batch size, and concurrency change requirements.

Model / Usage	Recommended GPU Size	Example Hardware
Pilot / 7B–8B quantized	16–24GB VRAM	RTX 4090 24GB, RTX 6000 Ada 48GB
13B–14B models	24–48GB VRAM	RTX 6000 Ada 48GB, NVIDIA L40S 48GB
30B–34B models	48GB+ VRAM	NVIDIA L40S 48GB, RTX 6000 Ada 48GB
70B–72B models	80GB+ or multi-GPU	NVIDIA A100 80GB, H100 80GB, 2× L40S
High concurrency / long context	2–8 GPUs, more VRAM, fast interconnect	H100/H200 class servers; H200 has 141GB VRAM

Minimum Server Resources for Better Performance



GPU: start from 24GB VRAM for pilot; prefer 48GB, 80GB, or 141GB for larger models and concurrent users.

- CPU: 24–64 cores depending on concurrency, preprocessing, embeddings, and gateway load.
 - RAM: 128GB minimum for serious internal use; 256GB+ preferred when hosting RAG and multiple services.
 - Storage: fast NVMe, usually 2TB+ for models, indexes, logs, and cached artifacts.
 - Network: 10GbE+ recommended between app, vector DB, storage, and inference server.
 - Operations: monitoring, GPU telemetry, backup, patching, capacity planning, and disaster recovery.
- 

Practical In-House Setup Example

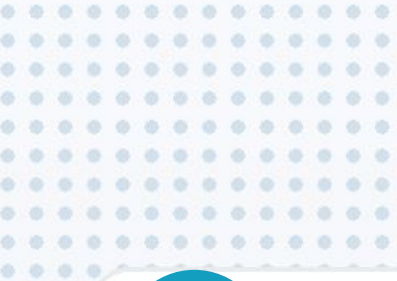
Component	Recommended Choice	Why
Operating system	Ubuntu Server / RHEL compatible Linux	Stable GPU driver and container ecosystem
Runtime	Docker + NVIDIA Container Toolkit	Simpler GPU-enabled deployment
Inference server	vLLM for throughput; Ollama for simple pilots; TGI as another production option	Choose based on model, concurrency, and team skill
Internal UI	Open WebUI or LibreChat	Company-only chat interface
Coding workflow	Tabby / Continue / Aider / OpenHands	Developer-focused AI assistance connected to approved model endpoint
Gateway and policy	LiteLLM + SSO + API gateway	Control access, budget, logs, and model routing

How to Limit In-House AI Usage to the Company



Deploy inside private network only; no public internet endpoint.

- Require VPN, zero-trust access, or private office network path.
 - Use SSO with Microsoft Entra ID / Active Directory / Keycloak and enforce MFA.
 - Use RBAC: developer, QA, DevOps, security, admin, and read-only roles.
 - Use API gateway rules: rate limit, quota, model allowlist, request size limit, and audit logging.
 - Restrict model access by team/project and block unknown external model endpoints.
 - Use DLP/secret scanning on prompts, uploaded files, and generated outputs where possible.
- 



9

AI in Software Development & Release Pipeline

AI should be inserted as assistive checkpoints around the existing SDLC, CI/CD, QA, security, and release controls — not as an uncontrolled release authority.



Software Delivery
AI Integration

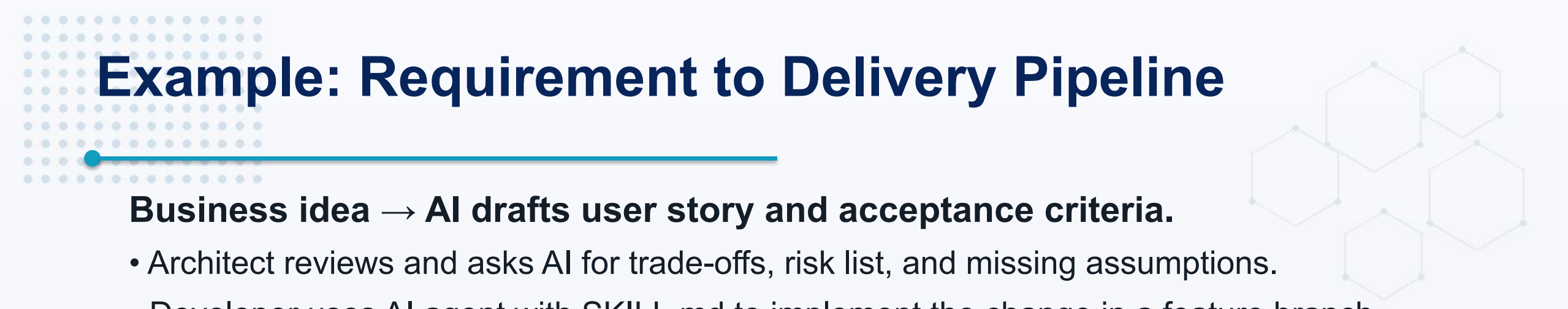


AI Integration Across the SDLC Pipeline

The AI assist layer should make each stage faster, but the gate remains human and deterministic.

Pipeline Stage	AI Integration Example	Required Control
Requirements / Backlog	Draft user stories, acceptance criteria, and clarification questions	Product owner review
Architecture / Design	Compare options, draft ADRs, list trade-offs and risks	Architecture review
Development	Generate scoped code, tests, refactors, and documentation using SKILL.md	Pull request + tests
QA / Testing	Generate test cases, negative scenarios, and regression checklist	QA sign-off
CI/CD / Release	Summarize failures, release notes, rollback plan, and deployment risks	Approval gate
Operations	Summarize logs, incidents, timeline, and runbook improvements	SRE / incident review

Example: Requirement to Delivery Pipeline



Business idea → AI drafts user story and acceptance criteria.

- Architect reviews and asks AI for trade-offs, risk list, and missing assumptions.
 - Developer uses AI agent with SKILL.md to implement the change in a feature branch.
 - QA uses AI to generate test scenarios, edge cases, and regression checklist.
 - CI validates build, tests, SAST/SCA, secrets scanning, container and IaC checks.
 - Release manager uses AI to draft release note, deployment checklist, and rollback plan.
 - No AI output bypasses review, CI, security scanning, or release approval.
- 

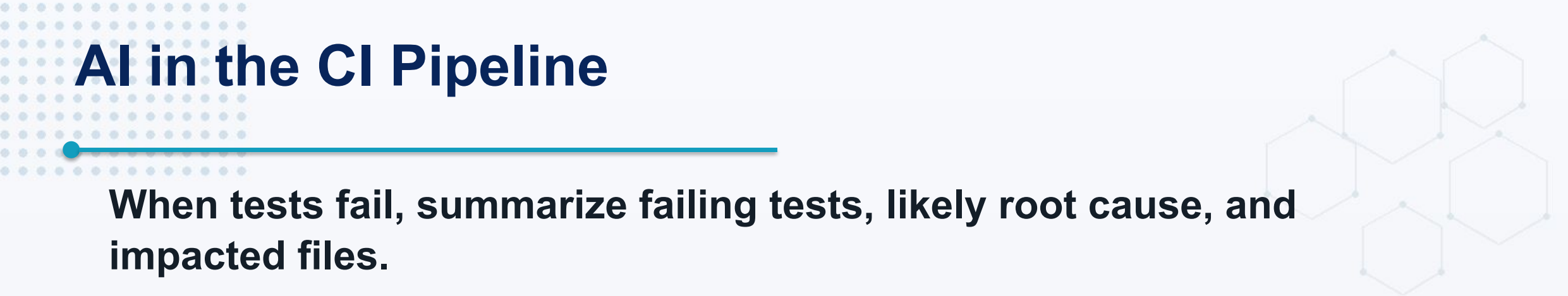
AI in the Pull Request Pipeline



Summarize changed files, affected modules, and potential blast radius.

- Generate reviewer checklist based on changed endpoints, database scripts, configuration, and security-sensitive code.
 - Suggest missing unit, integration, and contract tests.
 - Explain complex diffs for reviewers, especially in unfamiliar code areas.
 - Important: AI can help review, but it must not replace reviewer ownership.
- 

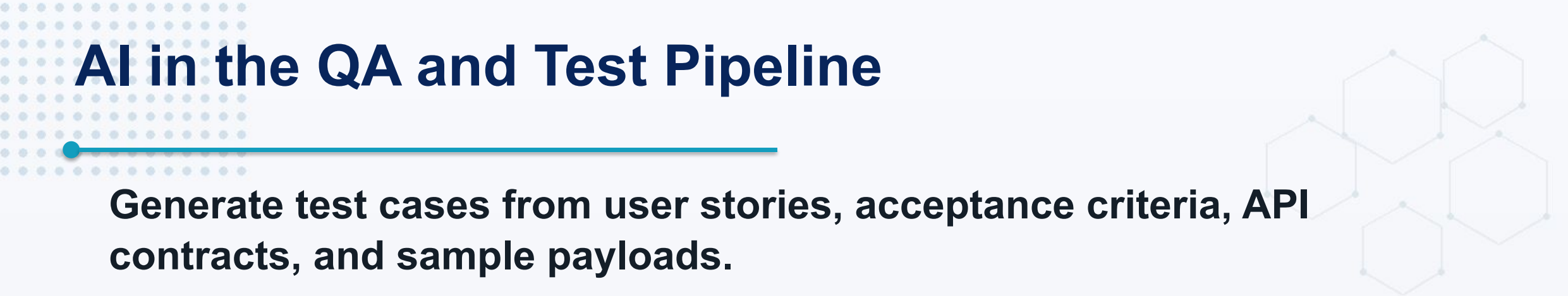
AI in the CI Pipeline



When tests fail, summarize failing tests, likely root cause, and impacted files.

- Group static-analysis findings and suggest remediation order.
 - Explain dependency, build, linting, or container scan failures in developer-friendly language.
 - Generate a short CI failure report for the PR comment.
 - Important: pass/fail must come from deterministic tools, not from AI judgment.
- 

AI in the QA and Test Pipeline



Generate test cases from user stories, acceptance criteria, API contracts, and sample payloads.

- Create negative tests, boundary values, invalid inputs, and permission scenarios.
 - Suggest missing regression tests from code diff or release scope.
 - Summarize bug reports and propose reproduction steps.
 - Generate synthetic test data, but never use real customer or production data without approval.
 - QA lead still owns coverage, risk assessment, and sign-off.
- 

AI in the Release Pipeline



Generate release notes from merged PRs, ticket IDs, and change summaries.

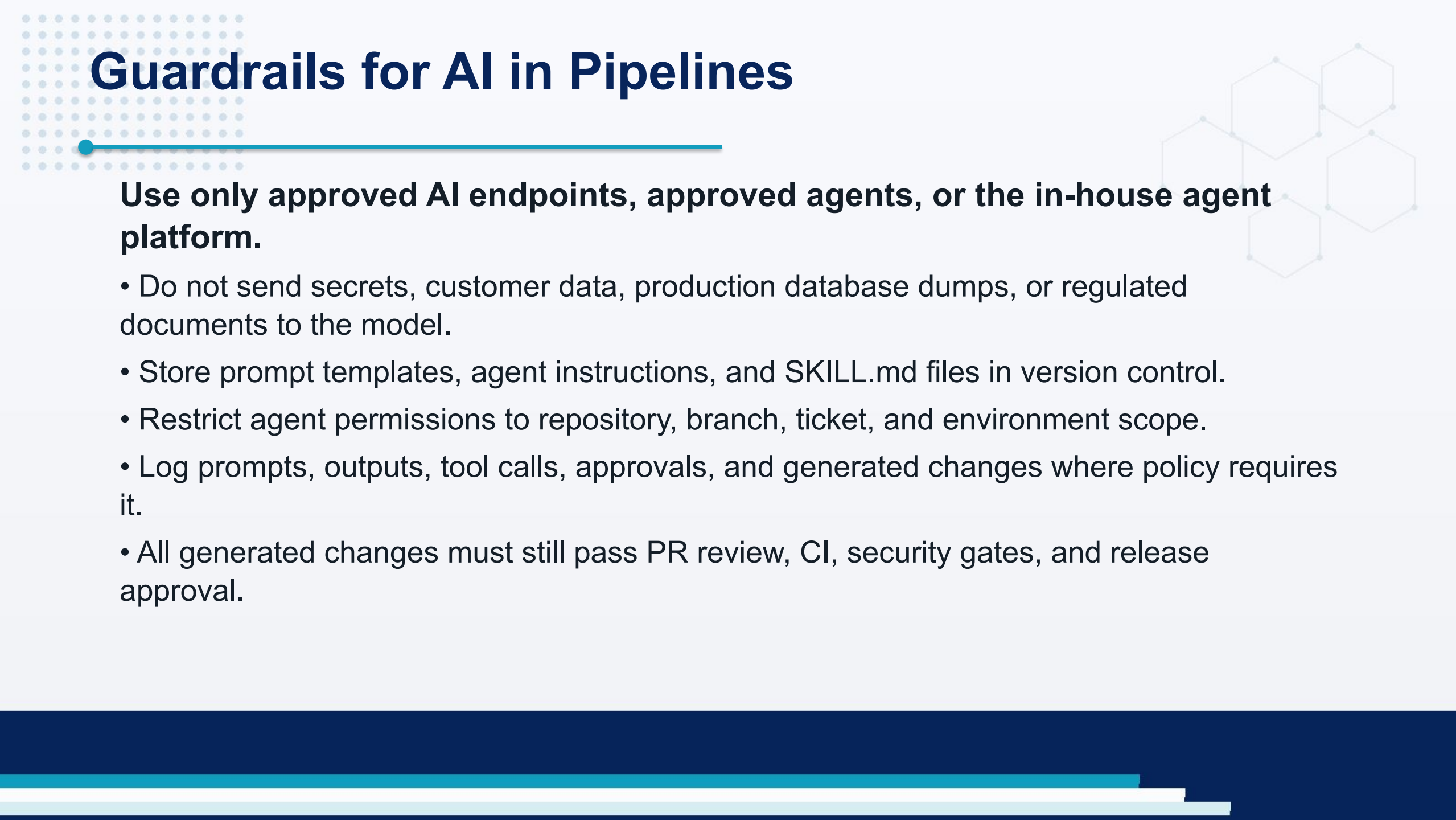
- Draft deployment checklist, migration risks, rollback plan, and smoke-test checklist.
 - Summarize feature flags, configuration changes, database changes, and known limitations.
 - Prepare stakeholder communication and support handover notes.
 - Release approval remains manual, auditable, and owned by the release authority.
- 

Example Release Flow with AI

AI should make release work clearer and faster, but every production decision must remain accountable.

Trigger	AI Assist	Gate
New PR opened	Summarize changes and impacted areas	Developer and reviewer approval
CI failed	Explain likely cause and suggest next investigation step	Tests must pass
Security scan found issue	Summarize severity and possible remediation	Security approval
Release candidate ready	Draft release notes, rollback plan, and smoke-test list	Release manager sign-off
Production incident	Summarize logs, timeline, and suspected impact	Incident commander approval

Guardrails for AI in Pipelines



Use only approved AI endpoints, approved agents, or the in-house agent platform.

- Do not send secrets, customer data, production database dumps, or regulated documents to the model.
- Store prompt templates, agent instructions, and SKILL.md files in version control.
- Restrict agent permissions to repository, branch, ticket, and environment scope.
- Log prompts, outputs, tool calls, approvals, and generated changes where policy requires it.
- All generated changes must still pass PR review, CI, security gates, and release approval.



9A. Sample AI Tool Names in the Pipeline

The following examples make the SDLC integration concrete by naming tools at each stage.



**Tools, Hosting
& Governance**



Sample Tools by Software Pipeline Stage

Stage	AI Usage	Sample Tools
Requirements	Summarize, split epics, draft acceptance criteria	ChatGPT, Claude, Copilot Chat, Azure DevOps, Jira
Design	Review architecture, ADRs, API contracts	ChatGPT, Claude, Mermaid, Structurizr, Backstage
Development	Code generation, refactor, explain, tests	GitHub Copilot, Codex, Claude Code, Tabby, Continue, Aider
Code Review	PR summary, risky diff detection, review checklist	Copilot PR summary, Codex review, Claude, Reviewdog
Security	SAST/SCA triage, fix suggestions, threat-model prompts	Semgrep, Snyk, GitHub Advanced Security, SonarQube, OWASP ZAP
QA	Generate test ideas, automate UI/API tests, analyze failures	Playwright, Cypress, Postman/Newman, TestRail, Zephyr, Allure

Sample Tools by Release and Operations Stage

Stage	AI Usage	Sample Tools
CI	Explain build failures, summarize test output, suggest fixes	GitHub Actions, Azure DevOps, GitLab CI, Jenkins, Buildkite
CD / GitOps	Review manifests, explain drift, summarize deployment risk	Argo CD, Flux, Helm, Kustomize, Terraform, Pulumi
Observability	Summarize logs, correlate errors, draft incident notes	Grafana, Prometheus, Datadog, New Relic, Sentry, OpenTelemetry
Release Notes	Summarize commits, tickets, PRs, risks, rollout plan	Jira, Azure DevOps, GitHub Releases, Slack, Microsoft Teams
Compliance	Evidence summary, control mapping, audit-ready notes	DefectDojo, Drata/Vanta-style tools, Jira, Confluence, SharePoint

Example: AI-Enabled Pull Request Pipeline

Step	Tool Example	AI Output
PR opened	GitHub / GitLab / Azure Repos	PR summary and changed-area explanation
Static analysis	SonarQube, Semgrep, Snyk, GitHub Advanced Security	Findings summary and remediation suggestions
Test review	GitHub Actions / Azure DevOps / GitLab CI	Failed-test explanation and likely root cause
Human review	Copilot Chat / Codex / Claude Code	Review checklist, edge cases, and questions for reviewer
Merge gate	Branch policy + required checks	AI cannot merge unless quality gates pass

Example: AI-Enabled QA Pipeline

Step	Tool Example	AI Output
Story analysis	Jira / Azure Boards + ChatGPT / Claude	Acceptance criteria gaps and test ideas
API testing	Postman, Newman, REST Assured	Generated API cases and negative scenarios
UI testing	Playwright, Cypress, Selenium	Draft test flows and locator review suggestions
Bug triage	TestRail, Zephyr, Allure, Sentry	Failure summary, repro steps, likely component owner
Regression scope	CI history + test management tool	Risk-based test selection suggestions

Example: AI-Enabled Release Pipeline

Step	Tool Example	AI Output
Change summary	GitHub Releases, Jira, Azure DevOps	Release notes and feature/fix summary
Deployment check	Argo CD, Flux, Helm, Terraform	Manifest drift and rollout risk summary
Security check	Snyk, Trivy, Gype, OWASP ZAP	Vulnerability summary and release-blocking issues
Go / No-Go	Slack / Teams + approval workflow	Decision brief with risks, rollback, and owner list
Post-release	Grafana, Datadog, Sentry, Prometheus	Incident summary, anomaly explanation, and follow-up tasks

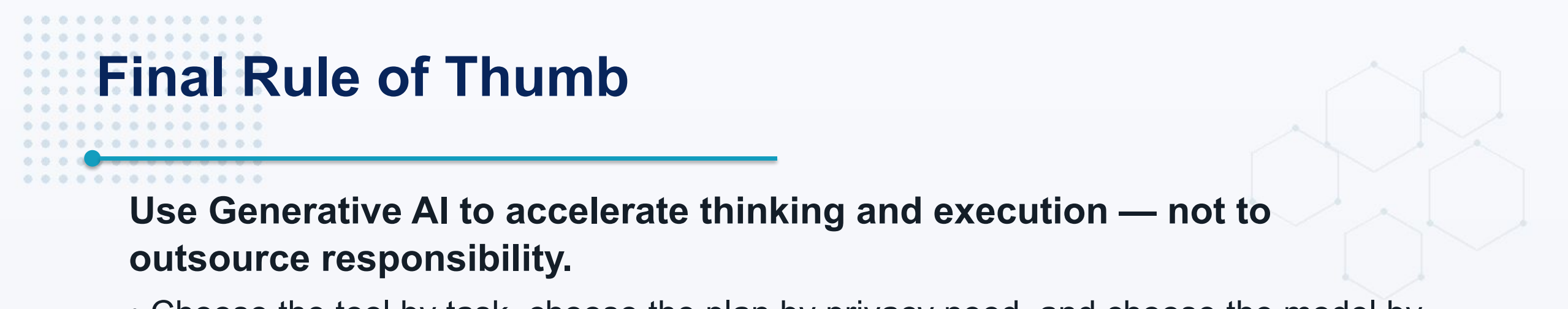
Rule for Pipeline AI Tools



AI can summarize, suggest, explain, and draft fixes.

- AI should not silently approve, merge, deploy, or override a failed security gate.
 - Every AI action should be traceable to a user, pipeline run, ticket, or pull request.
 - For company use, route tools through approved accounts, approved models, and approved network paths.
 - When possible, prefer deterministic tools for enforcement and AI for interpretation.
- 

Final Rule of Thumb



Use Generative AI to accelerate thinking and execution — not to outsource responsibility.

- Choose the tool by task, choose the plan by privacy need, and choose the model by risk and complexity.
 - Give the AI clear context, constrain it with team rules, and verify everything important.
 - The safest and most productive teams treat AI as a powerful junior collaborator with excellent speed, but imperfect judgment.
- 

References Used

Area	Reference
Prompt engineering	DeepLearning.AI: ChatGPT Prompt Engineering for Developers
AI prompting	DeepLearning.AI: AI Prompting for Everyone
OpenAI privacy	OpenAI business data / enterprise privacy documentation
GitHub Copilot	GitHub Docs: AI models, privacy FAQ, and policy updates
Anthropic / Claude	Anthropic enterprise, consumer terms, and Claude docs
OpenAI Codex	OpenAI Developers: Codex overview, models, and pricing docs

References Used — Added Tooling Sources

Area	Reference
Inference serving	vLLM official docs; Hugging Face Text Generation Inference docs; Ollama docs
Self-hosted UI / gateway	Open WebUI docs; LiteLLM docs
Self-hosted coding assistant	TabbyML official site and docs
GPU sizing anchors	NVIDIA L40S, A100, H100, and H200 product pages
Pipeline examples	GitHub Actions, Azure DevOps, GitLab CI, Jenkins, Argo CD, Flux, SonarQube, Semgrep, Snyk documentation



Thank you

